

Database schema

USERS TABLE

```
users
- id (uuid, primary key)
- email (text, unique)
- display_name (text)
- avatar_url (text)
- tier (text) – 'free', 'builder', 'operator', 'founder'
- archetype (text) – generated from questionnaire
- created_at (timestamp)
- last_active (timestamp)
- onboarding_complete (boolean)
```

XP TABLE

One row per user, three pools clearly separated.

```
user_xp
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- earned_xp (integer, default 0)
- purchased_xp (integer, default 0)
- rollover_xp (integer, default 0)
- reserved_xp (integer, default 0) – the lock mechanism
- daily_tokens_used (integer, default 0)
- monthly_tokens_used (integer, default 0)
- daily_reset_at (timestamp)
- monthly_reset_at (timestamp)
- xp_multiplier (float, default 1.0) – tier based
- last_updated (timestamp)
```

XP TRANSACTION LOG

Every deduction, earn, purchase, rollover recorded.

```
xp_transactions
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
```

```
- transaction_type (text) – 'earn', 'deduct', 'purchase', 'rollover', 'reserve', 'release'  
- amount (integer)  
- feature (text) – 'mentor_message', 'voice_session', 'image_upload' etc  
- tokens_consumed (integer)  
- model_used (text)  
- created_at (timestamp)
```

ROADMAPS TABLE

```
roadmaps  
- id (uuid, primary key)  
- user_id (uuid, foreign key → users)  
- title (text)  
- archetype (text)  
- status (text) – 'active', 'deleted', 'completed'  
- completion_percentage (float, default 0)  
- current_phase (integer, default 1)  
- total_phases (integer)  
- generated_at (timestamp)  
- deleted_at (timestamp, nullable)  
- reconfigure_available_at (timestamp, nullable) – deleted_at + 30 days  
- last_calibrated_at (timestamp, nullable)  
- roadmap_data (jsonb) – full roadmap structure
```

ROADMAP PHASES TABLE

```
roadmap_phases  
- id (uuid, primary key)  
- roadmap_id (uuid, foreign key → roadmaps)  
- phase_number (integer)  
- title (text)  
- description (text)  
- status (text) – 'locked', 'active', 'completed'  
- completion_percentage (float, default 0)  
- unlock_condition (text)  
- started_at (timestamp, nullable)  
- completed_at (timestamp, nullable)
```

TASKS TABLE

tasks

- id (uuid, primary key)
- roadmap_id (uuid, foreign key → roadmaps)
- phase_id (uuid, foreign key → roadmap_phases)
- user_id (uuid, foreign key → users)
- title (text)
- description (text)
- task_type (text) – 'daily', 'weekly', 'milestone', 'custom'
- status (text) – 'active', 'completed', 'overdue', 'skipped'
- difficulty (text) – 'low', 'medium', 'high'
- xp_reward (integer)
- due_date (timestamp, nullable)
- completed_at (timestamp, nullable)
- created_at (timestamp)
- is_boss_challenge (boolean, default false)
- reminder_at (timestamp, nullable)

STREAKS TABLE

user_streaks

- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- current_streak (integer, default 0)
- longest_streak (integer, default 0)
- last_active_date (date)
- shield_available (boolean, default false) – streak shield mechanic
- shield_used_at (timestamp, nullable)
- streak_broken_at (timestamp, nullable)

JOURNAL TABLE

journal_entries

- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- title (text, nullable)
- content (text)
- sentiment_score (float, nullable) – filled by background AI
- sentiment_label (text, nullable) – 'positive', 'neutral', 'negative', 'distressed'
- is_locked (boolean, default false)
- lock_type (text, nullable) – 'pin', 'biometric'
- tags (text array)
- created_at (timestamp)
- updated_at (timestamp)

MEMORY LAYER 1 — LIVE WINDOW

Per session, raw messages, never summarized.

```
memory_live_window
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- session_id (uuid)
- role (text) – 'user' or 'assistant'
- content (text)
- message_index (integer)
- created_at (timestamp)
```

MEMORY LAYER 2 — SESSION SUMMARY

Compressed active session context.

```
memory_session_summary
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- session_id (uuid)
- strategic_state (text)
- active_goals (jsonb)
- behavioral_patterns (jsonb)
- roadmap_progress (jsonb)
- recent_context (text)
- active_bottlenecks (jsonb)
- active_leverage_points (jsonb)
- emotional_state (text)
- token_count (integer)
- created_at (timestamp)
- updated_at (timestamp)
```

MEMORY LAYER 3 — PERSISTENT STRATEGIC MEMORY

Long term user intelligence, survives across all sessions.

```
memory_persistent
- id (uuid, primary key)
- user_id (uuid, foreign key → users, unique)
- user_stage (text)
- long_term_goals (jsonb)
- recurring_failures (jsonb)
- strengths (jsonb)
- weaknesses (jsonb)
```

```
- execution_patterns (jsonb)
- mentor_observations (jsonb)
- roadmap_state (text)
- consistency_score (float, default 0)
- discipline_trend (text)
- last_major_event (text)
- token_count (integer)
- updated_at (timestamp)
```

MEMORY LAYER 4 — EVENT MEMORY

Major behavioral moments that bypass decay.

```
memory_events
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- event_type (text) – 'streak_broken', 'milestone_completed', 'burnout_warning', 'phase_completed', 'discipline_collapse', 'major_breakthrough', 'consistency_recovery'
- severity (float)
- summary (text)
- impact (text)
- related_tags (text array)
- is_resolved (boolean, default false)
- created_at (timestamp)
```

MEMORY LAYER 5 — CACHED ANALYSIS

Saved expensive analysis so it isn't regenerated every session.

```
memory_cached_analysis
- id (uuid, primary key)
- user_id (uuid, foreign key → users, unique)
- bottleneck_analysis (jsonb)
- strength_analysis (jsonb)
- weakness_analysis (jsonb)
- leverage_points (jsonb)
- risk_assessment (jsonb)
- execution_profile (jsonb)
- generated_at (timestamp)
- invalidated_at (timestamp, nullable)
- trigger_for_regeneration (text, nullable)
```

MEMORY LAYER 6 — SESSION ARCHIVE

Compressed historical sessions, permanent record.

```
memory_session_archive
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- session_id (uuid)
- session_summary (text)
- major_events (jsonb)
- strategic_updates (jsonb)
- behavioral_changes (jsonb)
- token_cost (integer)
- archived_at (timestamp)
```

COMMUNITY TABLE

Progress sharing with privacy control.

```
community_posts
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- visibility (text) – 'public', 'limited', 'private'
- roadmap_title (text)
- completion_percentage (float)
- current_phase (integer)
- weekly_task_completion_rate (float)
- consistency_rating (float)
- current_streak (integer)
- discipline_score (float)
- archetype (text)
- momentum_score (float)
- message (text, nullable) – optional caption
- likes_count (integer, default 0)
- created_at (timestamp)
- updated_at (timestamp)
```

COMMUNITY INTERACTIONS

```
community_interactions
- id (uuid, primary key)
- post_id (uuid, foreign key → community_posts)
- user_id (uuid, foreign key → users)
- interaction_type (text) – 'like', 'encourage'
- created_at (timestamp)
```

CLOUD STORAGE REFERENCES

Tracks what's stored in Supabase storage buckets.

```
user_storage
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- file_type (text) – 'image', 'voice_recording'
- file_url (text)
- file_size_bytes (integer)
- storage_bucket (text)
- feature_context (text) – 'journal', 'roadmap', 'mentor'
- created_at (timestamp)
```

PERFORMANCE METRICS TABLE

Powers the performance tab and weekly/monthly reviews.

```
user_performance
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- period_type (text) – 'daily', 'weekly', 'monthly'
- period_start (date)
- period_end (date)
- tasks_completed (integer)
- tasks_total (integer)
- completion_rate (float)
- consistency_rating (float)
- momentum_score (float)
- discipline_score (float)
- xp_earned (integer)
- xp_spent (integer)
- mentor_sessions (integer)
- streak_days (integer)
- ai_summary (text, nullable)
- generated_at (timestamp)
```

QUESTIONNAIRE RESPONSES

Stored for archetype generation and roadmap logic.

```
onboarding_responses
- id (uuid, primary key)
- user_id (uuid, foreign key → users)
- responses (jsonb) – full questionnaire answers
- archetype_generated (text)
```

- roadmap_generated (boolean, default false)
- completed_at (timestamp)

Prompt;

You are building the backend database schema for a mobile-first AI mentorship and execution platform called Opus Devia. Your task is to implement the following schema into Supabase with full accuracy, but before implementing anything, audit the schema for errors, weaknesses, missing relationships, and logical gaps. Fix any issues you find and explain what you changed and why. Do not implement blindly.

AUDIT CHECKLIST — check these specifically before implementing:

Verify all foreign key relationships are correctly defined and will not cause orphaned records

Check that the 30-day roadmap deletion and reconfigure window logic is enforceable at the database level, not just the application level

Verify the XP system has no race condition vulnerabilities — the reserved_xp column must be atomic and the three pools (earned, purchased, rollover) must never produce a negative balance

Confirm the session_id threading across memory_live_window, memory_session_summary, and memory_session_archive is consistent and will correctly link all three layers to the same conversation

Check that the community_posts table correctly derives its metrics from live data rather than storing stale snapshots — flag if any columns will become outdated and suggest whether they should be computed views instead of stored columns

Verify the memory_cached_analysis table has a clear invalidation trigger path — the current schema has invalidated_at and trigger_for_regeneration columns but no enforcement mechanism. Flag this and suggest a database-level solution

Check that user_storage has size tracking that can enforce tier-based storage limits (50MB free, 500MB builder, 2GB operator, 10GB founder) and flag if this needs additional logic

Verify Row Level Security can be correctly applied to every table given the community section requires some public read access while all other tables require private user-only access

Check that the onboarding_responses table correctly gates roadmap generation

— a user should not be able to access roadmap features until `onboarding_complete` is true on the `users` table

Verify the `xp_transactions` log has enough information to reconstruct a user's full XP history for audit purposes and flag any missing fields

AREAS THAT NEED SPECIAL ATTENTION — do not skip these:

XP Race Conditions: The `reserved_xp` column is the lock mechanism that prevents a user from double-spending XP by firing multiple requests simultaneously. This must be implemented as an atomic database operation using Supabase's RPC functions, not a simple column update. Create a Supabase RPC function called `reserve_xp` that increments `reserved_xp` atomically and returns false if available XP is insufficient. Available XP is calculated as `earned_xp + purchased_xp + rollover_xp` minus `reserved_xp`.

Session Threading: The `session_id` that links memory layers 1, 2, and 6 is critical. A missing or mismatched `session_id` means the memory system breaks silently — the AI will lose context without throwing an error. Ensure `session_id` is a proper UUID generated at session start and passed consistently across `memory_live_window`, `memory_session_summary`, and `memory_session_archive`. Add a `sessions` table if needed to formally track session lifecycle.

Community Metrics Staleness: The `community_posts` table stores `completion_percentage`, `weekly_task_completion_rate`, `consistency_rating`, and `momentum_score` as static columns. These will become stale the moment a user completes a new task. Evaluate whether these should be a Postgres materialized view that pulls live from the `tasks`, `user_streaks`, and `user_performance` tables instead. If a materialized view is too complex for this stage, implement a database trigger that automatically updates `community_posts` whenever the underlying tables change.

Cache Invalidation: The `memory_cached_analysis` table has no enforcement mechanism for when to regenerate. Implement a Postgres trigger that sets `invalidated_at` to the current timestamp when specific events occur — specifically when a record is inserted into `memory_events` with `event_type` of `'streak_broken'`, `'discipline_collapse'`, `'phase_completed'`, or `'roadmap_shift'`. This ensures the cached analysis is automatically marked stale when the user's situation changes significantly.

Storage Limit Enforcement: The `user_storage` table tracks file sizes but has no running total. Add a Postgres function called `get_user_storage_total` that sums `file_size_bytes` for a given `user_id` and returns the total. Add a trigger that calls this

function before any insert into user_storage and rejects the insert if the total would exceed the user's tier limit. Tier limits are: free 52428800 bytes, builder 524288000 bytes, operator 2147483648 bytes, founder 10737418240 bytes.

Negative XP Prevention: The deduction logic must never produce a negative balance in any pool. Add a Postgres constraint to the user_xp table that enforces earned_xp >= 0, purchased_xp >= 0, and rollover_xp >= 0 at all times. The deduction order must always be rollover first, then purchased, then earned.

Onboarding Gate: Add a Postgres RLS policy that prevents any insert into roadmaps for a user where onboarding_complete is false on the users table. This must be enforced at the database level not just the application level.